Advanced Algorithms

Unit I

Introduction and Review of Sorting Algorithms

Dr. Ayush Kumar Agrawal

SoCS, UPES

Lecture Objectives

After this lecture, students will be able to:

- Understand the scope and importance of advanced algorithms
- Recall fundamental concepts from Design and Analysis of Algorithms
- Analyze the performance of sorting algorithms
- Differentiate sorting techniques based on complexity and properties

Why Advanced Algorithms?

- Modern problems involve massive data and complex constraints
- Classical algorithms are often insufficient
- Need for:
  - Optimal performance
  - Proven correctness
  - Scalability
- Advanced algorithms form the backbone of:
  - Artificial Intelligence
  - Big Data Analytics
  - Networking and Security
  - Scientific Computing

Prerequisite Knowledge

Students are expected to be familiar with:

- Asymptotic notations: O, Ω, Θ
- Basic sorting algorithms
- Recursion and divide-and-conquer paradigm
- Elementary data structures

Algorithm Analysis Review

Algorithm analysis focuses on:

- Time complexity
- Space complexity
- Worst-case, average-case, and best-case analysis

Why analysis matters:

- Machine-independent evaluation
- Predictability of performance
- Selection of optimal algorithms

Classification of Sorting Algorithms

Sorting algorithms can be classified based on:

- Comparison-based vs non-comparison-based
- Stable vs unstable sorting
- In-place vs out-of-place sorting
- Recursive vs iterative approaches

Comparison-Based Sorting

- Sorting decisions are made by comparing elements
- Lower bound of comparison-based sorting:

$$\Omega(n \log n)$$

- Examples:
  - Merge Sort
  - Quick Sort
  - Heap Sort

Merge Sort

- Divide-and-conquer based algorithm
- Divides array into halves recursively
- Merges sorted subarrays

Complexity:

- Time: $O(n \log n)$
- Space: $O(n)$

Quick Sort

- Based on partitioning strategy
- Pivot element selection is critical
- Widely used in practice

Complexity:

- Best/Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Heap Sort

- Uses binary heap data structure
- Does not require additional memory

Complexity:

- Time: $O(n \log n)$
- Space: $O(1)$

Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst
Merge Sort	$n \log n$	$n \log n$	$n \log n$
Quick Sort	$n \log n$	$n \log n$	n^2
Heap Sort	$n \log n$	$n \log n$	$n \log n$

Key Takeaways

- Sorting is a fundamental algorithmic problem
- Algorithm selection depends on constraints and data
- Complexity analysis guides optimal decision-making
- Advanced algorithms emphasize correctness and efficiency

Preview of Next Lecture

- Topological sorting
- Graph representations
- Introduction to Breadth First Search

Graph Fundamentals and Topological Sorting

After this lecture, students will be able to:

- Understand basic graph terminology and representations
- Distinguish between different types of graphs
- Explain the concept of topological sorting
- Analyze the correctness and complexity of topological sorting algorithms

Introduction to Graphs

A graph is a mathematical structure used to model pairwise relations.

- A graph is defined as $G = (V, E)$
- V represents the set of vertices
- E represents the set of edges

Graphs are widely used in:

- Computer networks
- Dependency analysis
- Scheduling problems
- Artificial intelligence

Types of Graphs

- Undirected graphs
- Directed graphs (Digraphs)
- Weighted and unweighted graphs
- Cyclic and acyclic graphs
- Connected and disconnected graphs

Graph Representations

Two standard ways to represent graphs:

- Adjacency Matrix
- Adjacency List

Choice of representation affects:

- Time complexity of algorithms
- Space requirements

Adjacency Matrix

- A two-dimensional matrix of size $|V| \times |V|$
- Entry $A[i][j] = 1$ if edge exists, else 0

Properties:

- Space complexity: $O(V^2)$
- Edge lookup in $O(1)$ time

Adjacency List

- Each vertex stores a list of adjacent vertices
- Efficient for sparse graphs

Properties:

- Space complexity: $O(V + E)$
- Iterating over neighbors is efficient

Directed Acyclic Graph (DAG)

- A directed graph with no cycles
- Important in dependency modeling

Applications:

- Task scheduling
- Compiler design
- Course prerequisite planning

Topological Sorting

Topological sorting is:

- A linear ordering of vertices
- For every directed edge (u, v), u appears before v

Key Property:

- Exists only for Directed Acyclic Graphs

Topological Sorting Algorithms

Two common approaches:

- Kahn's Algorithm (Indegree-based)
- Depth First Search (DFS)-based approach

Kahn's Algorithm

- Compute indegree of all vertices
- Insert vertices with indegree zero into a queue
- Remove vertices one by one and update indegrees

If all vertices are processed, graph is a DAG.

DFS-Based Topological Sorting

- Perform DFS on the graph
- Push vertex to stack after visiting all neighbors
- Pop stack to obtain topological order

Correctness of Topological Sorting

- DAG guarantees no circular dependencies
- Each vertex is placed only after its prerequisites
- DFS finishing time ensures correct ordering

Complexity Analysis

- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$

Efficient for large sparse graphs.

Key Takeaways

- Graph representation impacts algorithm efficiency
- Topological sorting solves dependency problems
- Only applicable to Directed Acyclic Graphs
- Foundational for advanced graph algorithms

Preview of Next Lecture

- Breadth First Search (BFS)
- Shortest path in unweighted graphs
- Correctness proof of BFS

Breadth First Search and Shortest Paths

After this lecture, students will be able to:

- Understand the working principle of Breadth First Search
- Apply BFS for graph traversal
- Compute shortest paths in unweighted graphs
- Analyze correctness and complexity of BFS

Motivation for BFS

- Many problems require finding minimum distance or minimum hops
- Examples:
  - Shortest path in unweighted networks
  - Broadcasting in networks
  - Social network analysis
- Breadth First Search provides an optimal solution for such problems

Breadth First Search (BFS)

Breadth First Search is a graph traversal algorithm that:

- Explores vertices level by level
- Visits all neighbors before going deeper
- Uses a queue data structure

BFS Algorithm – High Level

- Initialize all vertices as unvisited
- Mark the source vertex as visited
- Insert source into a queue
- While the queue is not empty:
  - Dequeue a vertex
  - Visit all unvisited adjacent vertices
  - Mark them visited and enqueue

BFS Traversal Example

- BFS explores graph in increasing order of distance
- Nodes at distance k are visited before nodes at distance $k + 1$

This property makes BFS suitable for shortest path computation.

Shortest Path in Unweighted Graphs

- Each edge has equal weight
- Shortest path equals minimum number of edges
- BFS guarantees shortest path from source to all reachable vertices

Distance and Parent Arrays

During BFS, we maintain:

- Distance array:
  - Stores minimum distance from source
- Parent array:
  - Stores predecessor for path reconstruction

Correctness of BFS

- BFS explores vertices in non-decreasing order of distance
- When a vertex is first discovered, the shortest path to it is found
- Proof based on induction over BFS levels

Proof Sketch

- Base Case:
  - Source vertex has distance zero
- Inductive Step:
  - Assume shortest paths to all vertices at distance k
  - BFS discovers vertices at distance $k + 1$ via edges from level k

Complexity Analysis

- Time Complexity: $O(V + E)$
- Space Complexity:
  - $O(V)$ for visited, distance, and parent arrays
  - Queue storage

Applications of BFS

- Shortest path in unweighted graphs
- Level-order traversal
- Detecting bipartite graphs
- Connected components in graphs

Limitations of BFS

- Not suitable for weighted graphs
- Cannot handle negative edge weights
- Requires additional memory for queue

Key Takeaways

- BFS is a fundamental graph traversal algorithm
- Guarantees shortest path in unweighted graphs
- Correctness is based on level-wise exploration
- Forms the basis for advanced graph algorithms

Preview of Next Lecture

- Dijkstra's algorithm
- Shortest path in edge-weighted graphs
- Greedy strategy and correctness

Dijkstra's Algorithm for Shortest Paths

After this lecture, students will be able to:

- Understand shortest path problems in weighted graphs
- Explain the greedy strategy behind Dijkstra's algorithm
- Apply Dijkstra's algorithm to compute shortest paths
- Analyze correctness and time complexity of the algorithm

Motivation

- Real-world networks have weighted edges
- Examples:
  - Road networks with distances
  - Communication networks with delays
  - Cost-based routing problems
- Breadth First Search fails when edge weights differ
- Dijkstra's algorithm efficiently solves this problem

Problem Definition

Given:

- A graph $G = (V, E)$
- Non-negative edge weights
- A source vertex s

Goal:

- Compute shortest path from s to all other vertices

Key Assumption

- All edge weights must be non-negative
- Algorithm fails in presence of negative weight edges

This assumption is crucial for correctness.

Greedy Strategy

- Maintain a set of vertices with finalized shortest distances
- Repeatedly select the vertex with minimum tentative distance
- Relax all outgoing edges from the selected vertex

Dijkstra's Algorithm – High Level

- Initialize distance of source as zero
- Set distance of all other vertices to infinity
- Insert all vertices into a priority queue
- While priority queue is not empty:
  - Extract vertex with minimum distance
  - Relax all adjacent edges

Edge Relaxation

For an edge (u, v) with weight $w(u, v)$:

$$\text{if } dist[v] > dist[u] + w(u, v)$$

$$\text{then } dist[v] = dist[u] + w(u, v)$$

Relaxation improves the estimate of shortest distance.

Data Structures Used

- Array for distance values
- Priority Queue (Min Heap)
- Adjacency list representation

Correctness of Dijkstra's Algorithm

- Greedy choice is safe due to non-negative weights
- Once a vertex is selected, its shortest distance is finalized
- No shorter path can exist through unvisited vertices

Proof Sketch

- Assume a shorter path exists after vertex is finalized
- Such a path must pass through a vertex with greater distance
- Contradicts greedy selection of minimum distance vertex

Complexity Analysis

- Using adjacency list and binary heap:
  - Time Complexity: $O((V + E) \log V)$
- Using array-based implementation:
  - Time Complexity: $O(V^2)$

Comparison: BFS vs Dijkstra

Feature	BFS	Dijkstra
Edge Weights	Equal	Non-negative
Shortest Path	Yes	Yes
Data Structure	Queue	Priority Queue
Time Complexity	$O(V + E)$	$O((V + E) \log V)$

Applications

- Network routing protocols
- GPS navigation systems
- Resource optimization problems

Limitations

- Cannot handle negative edge weights
- Bellman–Ford algorithm is required in such cases

Key Takeaways

- Dijkstra's algorithm is a greedy algorithm
- Efficient for weighted graphs with non-negative edges
- Correctness depends on greedy choice property
- Forms basis for advanced shortest path algorithms

Preview of Next Lecture

- Depth First Search (DFS)
- Strongly Connected Components
- Role of DFS in graph analysis

Depth First Search and Applications

After this lecture, students will be able to:

- Understand the working of Depth First Search
- Perform DFS traversal on graphs
- Apply DFS to solve graph problems
- Analyze correctness and complexity of DFS

Motivation for DFS

- Many problems require exploring all possible paths
- Examples:
  - Maze solving
  - Cycle detection
  - Topological sorting
- Depth First Search provides a systematic approach

Depth First Search (DFS)

Depth First Search is a graph traversal algorithm that:

- Explores as deep as possible before backtracking
- Uses recursion or an explicit stack
- Works for both directed and undirected graphs

DFS Algorithm – High Level

- Mark the current vertex as visited
- Visit an unvisited adjacent vertex recursively
- Backtrack when no unvisited neighbors remain

Recursive and Iterative DFS

- Recursive DFS uses call stack
- Iterative DFS uses an explicit stack
- Both produce the same traversal order (depending on adjacency list)

DFS Tree and Forest

- DFS produces a DFS tree for connected graphs
- DFS forest for disconnected graphs
- Tree edges represent discovery relationships

Edge Classification in DFS

Edges can be classified as:

- Tree edges
- Back edges
- Forward edges
- Cross edges

Applications of DFS

- Cycle detection in graphs
- Topological sorting
- Strongly connected components
- Path existence queries

Correctness of DFS

- DFS systematically explores all reachable vertices
- Recursive calls ensure complete traversal
- Each edge is explored at most once

DFS Discovery and Finish Times

- Discovery time: when a vertex is first visited
- Finish time: when exploration of a vertex is complete
- Used in advanced graph algorithms

Complexity Analysis

- Time Complexity: $O(V + E)$
- Space Complexity:
  - $O(V)$ for visited array and recursion stack

DFS vs BFS

Feature	DFS	BFS
Traversal Strategy	Depth-first	Level-order
Data Structure	Stack / Recursion	Queue
Shortest Path	No	Yes (unweighted)
Memory Usage	Lower	Higher

Limitations of DFS

- Does not guarantee shortest path
- Deep recursion may cause stack overflow

Key Takeaways

- DFS is a fundamental graph traversal technique
- Enables powerful graph analyses
- Discovery and finish times are crucial concepts
- Basis for SCC and other advanced algorithms

Preview of Next Lecture

- Strongly Connected Components
- Kosaraju's and Tarjan's algorithms
- Amortized analysis examples

Strongly Connected Components and Amortized Analysis

After this lecture, students will be able to:

- Understand the concept of strongly connected components
- Apply algorithms to compute SCCs
- Analyze correctness and complexity of SCC algorithms
- Understand the basics of amortized analysis

Motivation for SCC

- Directed graphs often contain cycles
- Need to identify maximal groups of mutually reachable vertices
- SCCs simplify complex directed graphs

Applications include:

- Compiler optimization
- Deadlock detection
- Program flow analysis

Strongly Connected Components

- A strongly connected component is a maximal subgraph
- Every vertex is reachable from every other vertex
- Defined only for directed graphs

Why SCCs Matter

- Collapsing SCCs forms a Directed Acyclic Graph
- Enables topological sorting on component graph
- Reduces complexity of many graph problems

Kosaraju's Algorithm

Key idea:

- Use Depth First Search twice
- Reverse graph edges
- Use finishing times from first DFS

Kosaraju's Algorithm – Steps

- Perform DFS on original graph
- Record vertices in order of decreasing finish time
- Reverse all edges of the graph
- Perform DFS in recorded order
- Each DFS tree forms one SCC

Correctness of Kosaraju's Algorithm

- Finish times capture reachability structure
- Reversing edges isolates SCCs
- DFS explores exactly one SCC at a time

Tarjan's Algorithm

- Single-pass DFS based algorithm
- Uses discovery time and low-link values
- Uses stack to track active vertices

Low-Link Values

- Low-link value of a vertex represents
- Minimum discovery time reachable
- Includes back edges

Used to detect root of SCC.

Complexity Analysis of SCC Algorithms

- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$

Efficient even for large graphs.

Transition to Amortized Analysis

- Worst-case analysis may be too pessimistic
- Some operations are expensive but rare
- Amortized analysis provides average cost per operation

Amortized Analysis

- Analyzes the cost of a sequence of operations
- Guarantees average cost per operation
- No probability involved

Methods of Amortized Analysis

- Aggregate method
- Accounting method
- Potential method

Example: Dynamic Array

- Insert operation occasionally triggers resizing
- Individual insert may cost $O(n)$
- Amortized cost per insert is $O(1)$

Why Amortized Analysis is Important

- Explains real-world efficiency of algorithms
- Essential for advanced data structures
- Used in algorithm design and proofs

Key Takeaways

- SCCs reveal structural properties of directed graphs
- Kosaraju's and Tarjan's algorithms are efficient
- Amortized analysis complements worst-case analysis
- Unit I builds foundation for advanced graph algorithms

Unit I Completion

- Sorting and graph fundamentals
- Shortest path algorithms
- DFS and SCCs
- Introduction to amortized analysis